Malware – Lab

CS930 Foundations of Cyber Security

University of Warwick

Work individually, completing the tasks in order.

Tasks 1–3: Complete these tasks in the browser VM at `https://warwick-cs930.github.io/labs/lecture-05/`. This is a **deliberately altered teaching environment** containing fictional files. Work in the prepared investigation directory. **Restart lab** or reloading the page restores the starting image and discards guest changes.
Tasks 4–7: Download `lecture-05-lab-files.zip` and complete these tasks on your lab machine.

Task 1: Investigate a missing file

A colleague says that `collector.log` should be in the current directory. Investigate without changing any files. Run one command per line and stay in the browser terminal's starting directory.

1. In the browser terminal, run:

```
pwd
ls
```

   Try `ls` with different options, such as `-l`, `-la` and `-a`, in the VM and on your lab machine. How does the `ls` command in the VM look different from `ls` on the lab machine? Does `collector.log` appear in any of the VM's listings?

2. Back in the browser terminal, run:

```
cat collector.log
```

   What does the result tell you about whether the file exists and can be read? Why might `cat` read a file that `ls` does not show? Could the usual dotfile rule explain this filename?

Check your understanding: Does a missing entry in a directory listing prove that the file is absent? Which of your observations challenges that conclusion?

Task 2: Compare the listing with BusyBox

BusyBox provides several Linux utilities in one executable. In the browser VM, use this form to run one of its utilities directly:

```
/bin/busybox COMMAND ARGUMENTS
```

Replace `COMMAND` with a utility name, such as `ls`, and `ARGUMENTS` with its options and paths. The full path selects the supplied BusyBox executable instead of asking the shell to find a program named `ls`.

Trust assumption: for this investigation, the supplied `/bin/busybox` executable and the guest kernel are intact.

1. Run these commands in the browser terminal:

   ```
   ls -1
   /bin/busybox ls -1 .
   ```

   Both request one entry per line; `.` means the current directory. Compare the filenames. Which command shows `collector.log`? Do both show `welcome.txt`?

2. Compare the detailed listings:

   ```
   ls -la
   /bin/busybox ls -la .
   ```

   Does adding `-a` make the two listings agree? How do these results help distinguish the usual dotfile rule from a command suppressing an entry?

3. Read an ordinary file for comparison:

   ```
   cat welcome.txt
   ```

   Together with your earlier `cat collector.log` result, which listing agrees with direct access to the two files? Is the discrepancy affecting every entry or a particular entry? What still needs investigating to explain its cause?

Check your understanding: Why is the intact BusyBox assumption needed for this comparison? Would two different commands necessarily give trustworthy evidence on a compromised system?

Task 3: Find the cause and test a repair

Use your findings from Tasks 1–2 to explain why the normal listing omits the file, then make it report the directory correctly. Preserve the program as you found it and the investigation files.

Your workspace is `/home/cs930/investigation`. The VM stores this path in an **environment variable** named `LAB_ROOT`. Writing `$LAB_ROOT` in a shell command inserts that path; for example, `$LAB_ROOT/files` refers to the directory where you began the investigation. Make changes only inside this workspace, and preserve the supplied `/bin/busybox` executable.

1. Find which executable runs when you enter `ls`. Identify its path, explain how the shell selects it, and read its source using `cat PATH`, replacing `PATH` with the path you found. Which operations explain the difference between `ls` and the direct BusyBox listing?

2. Describe two possible ways to correct the normal listing. Compare what each changes, how it preserves evidence and any limitations. Choose one and explain why you prefer it.

3. Before changing anything, compare `ls -1` with `/bin/busybox ls -1 .` and read `collector.log` and `welcome.txt` again. Preserve a copy of the selected program and compare its contents with the original. Predict how the listings should change after your repair and which file contents should stay the same.

4. Apply your chosen change and repeat those listings and reads. Check that the normal listing now includes both files, their contents match your earlier observations, and the original program is preserved. Investigate any failed check. Explain what the repair fixes and what your tests cannot establish about the rest of the system.

Command reference. Choose the commands and paths needed for your approach; the entries below are syntax references, not a sequence to execute.

Command	Meaning
`command -v COMMAND`	Report how the shell resolves a command.
`printf '%s\n' "$PATH"`	Display the command-search directories, in order.
`printf '%s\n' "$LAB_ROOT"`	Display the investigation workspace path.
`cp SOURCE DESTINATION`	Copy a file; choose a new destination to preserve existing evidence.
`rm FILE`	Remove the named file.
`hash -r`	Clear remembered command locations after a relevant change.

The small guest does not support `mv`. In shell source, `case` selects a matching branch, `;;` ends it, and a `while` loop repeats its body. Quotes around an expanded path keep it as one argument. Ask for a syntax hint if needed; you do not need to write a shell program.

Check your understanding: Which part of the system did your repair change? Would your evidence justify the same conclusion if the kernel were also compromised?

Optional: predict whether your repair survives a fresh login shell. Run `exit`; the guest resumes in the same VM. Repeat your decisive tests and explain the scope. This differs from **Restart lab** or reloading the page, which resets the disk image.

Task 4: Match known malware signatures

Continue in a terminal on the lab machine. Download and extract `lecture-05-lab-files.zip` and work in the extracted folder. Use Python 3.8 or later; if `python --version` reports a suitable version, you may use `python` throughout.

```
python3 setup_lab.py
python3 check_lab.py --part signatures
```

Setup should print `LAB READY`; the checks initially report an unfinished function. The six `workspace/samples/sample-a.bin` to `sample-f.bin` files contain **inert, non-executable teaching bytes**. The activity logs are fictional. This detector reads supplied data: it does not run malware, encrypt files or perform the logged deletions. Leave the fixtures and support code unchanged.

Open `malware_scan.py`. Complete `signature_matches(data, signatures)`. It receives sample bytes and a dictionary mapping signature labels to byte patterns. Return a **sorted list of labels** whose nonempty patterns occur anywhere in the sample; ignore empty patterns. This is exact byte-pattern matching, not matching a filename or hashing the whole file.

1. Predict the result for each sample below with the same signature dictionary. Explain whether renaming a sample would affect the result.

   ```
   signatures = {"demo": b"ABCD"}
   first = b"START-ABCD-END"
   second = b"START-ABCE-END"
   ```

   The `b` prefix makes a bytes literal. The runner decodes `workspace/signatures.json` into this dictionary form for you.

2. Implement the function using a loop over `signatures.items()`. Python's `in` operator tests whether one byte sequence occurs within another; `sorted(labels)` sorts a collection of labels. Return all matching labels, not only the first.

3. Rerun the signature checks. Explain and correct any failure. The full scan will run after Task 5's function is complete.

Check your understanding: What would a signature match establish? Why would no match fail to establish that a sample is harmless?

Task 5: Detect ransomware-like activity

The fictional event log records what each process did to files. Complete `affected_files(events)` in `malware_scan.py`.

The runner supplies events for **one process in one time window**, already in chronological order. Each event is a dictionary, for example:

```
{"time": 2, "pid": 101, "action": "READ_SOURCE",
 "file": "report.txt"}
```

`time` is seconds since the log began; `pid` identifies the process. For file actions, `file` names the **original file**, including when the action concerns its companion copy.

Action	Recorded activity
`READ_SOURCE`	Read an original file.
`WRITE_LOCKED_COPY`	Write its companion `.locked` copy; this alone does not prove encryption.
`DELETE_SOURCE`	Delete the original file.
`WRITE_COPY`	Write an ordinary copy.
`STOP_BACKUPS`	Stop backups; `file` is `None`. This is context, not a required part of the rule.

Return a **set of original file identifiers** with a `READ_SOURCE`, then `WRITE_LOCKED_COPY`, then `DELETE_SOURCE` in that order for the same file. Other events may intervene; repeated actions do not reset progress. Count each file once. The runner alerts when at least **three distinct files** complete this sequence within a rolling **10-second window**, including both endpoints. Parsing, grouping, time windows and reporting are supplied.

1. Predict whether a file qualifies when the three actions occur in order, when deletion precedes the write, and when other files' events appear between them. A read of one file and deletion of another must not form a sequence.

2. Implement a loop that tracks progress separately for each file and collects completed identifiers in a set. `set()` creates an empty set; `.add(...)` records a unique identifier. Read fields with `event["action"]` and `event["file"]`. Ignore unrelated actions; do not treat a process name or `STOP_BACKUPS` as a substitute for the sequence.

3. Run the checks, then scan the prepared evidence:

   ```
   python3 check_lab.py
   python3 malware_scan.py
   ```

   The scanner saves `findings.json`; if that name exists, it chooses a numbered name. The `Saved:` line identifies the generated report. Unfinished code or unreadable evidence is an error, never a clean result.

Check your understanding: Why count distinct files rather than event lines? What does the ordered pattern suggest, and what does it fail to prove?

Task 6: Investigate the alerts and missed cases

Interpret signature matches and behaviour alerts separately.

1. Inspect the signature and behaviour results for cases A–F (PIDs 101–106). Compare them with `workspace/events.json`, then read `workspace/case-notes.txt` for the supplied context. For each case, distinguish the observed pattern from a judgement about intent.

2. Compare A and B: the samples contain identical bytes under different names. Explain why their signature results agree and how their activity logs change your assessment. Find a case with suspicious behaviour but no known signature. Would requiring `STOP_BACKUPS` cause a missed case?

3. Identify an authorised activity that triggers the behaviour rule and explain why it is a false alarm in this case. Compare it with the backup process that preserves originals. Neither a process name nor a missing signature is proof of safety.

4. Examine F's event times. Predict what a 60-second window will change, then run:

   ```
   python3 malware_scan.py --window 60 --output wider-window-findings.json
   ```

   Compare the two reports, using their actual saved names. Explain why widening the window can find slower activity and can also collect unrelated legitimate actions. Does the new result prove encryption or malicious intent?

Trust assumptions. Treat the supplied event log, sample-to-process mapping and case notes as reliable classroom evidence. In a real incident, their origin, completeness and integrity need checking; a compromised system may conceal activity.

Check your understanding: Which cases show a limit of known signatures, and which show a limit of this behaviour rule? Why should the two outputs inform an investigation rather than become a single "safe or malware" verdict?

Task 7: Assess the incident and clean up

Consider which cases warrant escalation and why, using the signature or activity evidence and the available context. What would you investigate next, and what remains uncertain?

Run:

```
python3 setup_lab.py --clean
```

Cleanup removes recognised fixtures and preserves your code and generated reports. If it reports unexpected personal files, move them outside `workspace/` and retry. Setup restores fixtures while preserving code edits; a fresh ZIP in another folder restores the starter.